

Fast Convolution; Strassen's Method

1 Fast Convolution

- reduction to subquadratic time
- polynomial evaluation at complex roots of unity
- interpolation via evaluation at complex roots of unity

2 The Master Method to Solve Recurrences

- a recursive matrix-matrix multiplication
- the master method

3 Strassen's method

- the steps in the method
- reduction to subcubic time

CS 401/MCS 401 Lecture 9
Computer Algorithms I
Jan Vershelde, 8 July 2026

Fast Convolution; Strassen's Method

1 Fast Convolution

- reduction to subquadratic time
- polynomial evaluation at complex roots of unity
- interpolation via evaluation at complex roots of unity

2 The Master Method to Solve Recurrences

- a recursive matrix-matrix multiplication
- the master method

3 Strassen's method

- the steps in the method
- reduction to subcubic time

computing convolutions

Input: $a = (a_0, a_1, \dots, a_{n-1})$, $b = (b_0, b_1, \dots, b_{n-1})$.

Output: $c = a \star b$, the convolution of a with b .

Example: $n = 4$, consider:

$$\begin{array}{cccccccc} & a_0b_0 & a_0b_1 & a_0b_2 & a_0b_3 & & & \\ & & a_1b_0 & a_1b_1 & a_1b_2 & a_1b_3 & & \\ & & & a_2b_0 & a_2b_1 & a_2b_2 & a_2b_3 & \\ + & & & & a_3b_0 & a_3b_1 & a_3b_2 & a_3b_3 \\ \hline c_0 & c_1 & c_2 & c_3 & c_4 & c_5 & c_6 & \end{array}$$

The general formula for the k -th coefficient in the convolution:

$$c_k = \sum_{i=\max(0, k-n+1)}^{\min(n-1, k)} a_{k-i}b_i.$$

applying divide and conquer

View the input $a = (a_0, a_1, \dots, a_{n-1})$ and $b = (b_0, b_1, \dots, b_{n-1})$ as coefficients of polynomials $A(x)$ and $B(x)$ respectively.

Compute $c = a \star b$ by polynomial evaluation and interpolation:

- 1 Choose $2n$ values $x_1, x_2, \dots, x_{2n}$. Evaluating A and B at those values gives $A(x_1), A(x_2), \dots, A(x_{2n}), B(x_1), B(x_2), \dots, B(x_{2n})$.
- 2 Multiply the $2n$ evaluations: $C(x_j) = A(x_j) \cdot B(x_j), j = 1, 2, \dots, 2n$. Evaluation commutes with computing the product of A and B .
- 3 Because the degree of C is $2n - 1$, C has $2n$ coefficients, which can be recovered from the $2n$ values $C(x_1), C(x_2), \dots, C(x_{2n})$ by polynomial interpolation.

Steps 2 runs in $O(n)$, but step 1 and 3 are both $O(n^2)$, for any x_i .

Lagrange interpolation

Given are $n + 1$ interpolation points (x_i, f_i) , $i = 0, 1, \dots, n$, where for all $i \neq j$: $x_i \neq x_j$. The Lagrange interpolating polynomial has the form

$$p(x) = \ell_0(x)f_0 + \ell_1(x)f_1 + \dots + \ell_n(x)f_n,$$

where

$$\ell_i(x_j) = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} \quad \ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \left(\frac{x - x_j}{x_i - x_j} \right).$$

In this form, we have that $p(x_i) = f_i$, $i = 0, 1, \dots, n$.

Exercise 1:

Write a pseudo code description for the computation of $p(x)$.
Prove that the algorithm to compute $p(x)$ runs in $O(n^2)$ time.

Fast Convolution; Strassen's Method

1 Fast Convolution

- reduction to subquadratic time
- **polynomial evaluation at complex roots of unity**
- interpolation via evaluation at complex roots of unity

2 The Master Method to Solve Recurrences

- a recursive matrix-matrix multiplication
- the master method

3 Strassen's method

- the steps in the method
- reduction to subcubic time

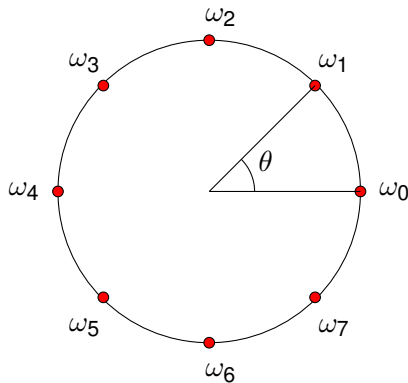
complex roots of unity

Consider the 8-th complex roots of unity:

$$\omega_1 = e^{2\pi/8} = \cos(2\pi/8) + \sin(2\pi/8)i, \quad i = \sqrt{-1},$$

$$\omega_k = e^{2k\pi/8}, \quad k = 0, 1, \dots, 7.$$

Let $\theta = 2\pi/8$:



squaring n -th roots of unity

Exercise 2:

For $n > 0$, show that the squares of the $2n$ complex roots of unity are the n complex roots of unity of $x^n - 1$.

evaluation at complex roots of unity

$$\begin{aligned}A(x) &= a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4 + a_5x^5 + a_6x^6 + a_7x^7 \\&= a_0 + a_2x^2 + a_4x^4 + a_6x^6 + a_1x + a_3x^3 + a_5x^5 + a_7x^7 \\&= a_0 + a_2x^2 + a_4x^4 + a_6x^6 + x(a_1 + a_3x^2 + a_5x^4 + a_7x^6)\end{aligned}$$

$$A_{\text{even}}(z) = a_0 + a_2z + a_4z^2 + a_6z^3$$

$$A_{\text{odd}}(z) = a_1 + a_3z + a_5z^2 + a_7z^3$$

We have thus $A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2)$. Consider:

$$A(\omega_0) = A_{\text{even}}(\omega_0) + \omega_0 A_{\text{odd}}(\omega_0)$$

$$A(\omega_1) = A_{\text{even}}(\omega_2) + \omega_1 A_{\text{odd}}(\omega_2)$$

$$A(\omega_2) = A_{\text{even}}(\omega_4) + \omega_2 A_{\text{odd}}(\omega_4)$$

$$A(\omega_3) = A_{\text{even}}(\omega_6) + \omega_3 A_{\text{odd}}(\omega_6)$$

$$A(\omega_4) = A_{\text{even}}(\omega_0) + \omega_4 A_{\text{odd}}(\omega_0), \text{ as } (\omega_4)^2 = e^{2 \cdot (2.4\pi/8)} = \omega_0$$

$$A(\omega_5) = A_{\text{even}}(\omega_2) + \omega_5 A_{\text{odd}}(\omega_2), \text{ as } (\omega_5)^2 = e^{2 \cdot (2.5\pi/8)} = \omega_2$$

$$A(\omega_6) = A_{\text{even}}(\omega_4) + \omega_6 A_{\text{odd}}(\omega_4), \text{ as } (\omega_6)^2 = e^{2 \cdot (2.6\pi/8)} = \omega_4$$

$$A(\omega_7) = A_{\text{even}}(\omega_6) + \omega_7 A_{\text{odd}}(\omega_6), \text{ as } (\omega_7)^2 = e^{2 \cdot (2.7\pi/8)} = \omega_6$$

deriving a recurrence

The evaluation of $A(x)$ at $2n$ points is reduced to

- 1 evaluation of $A_{\text{even}}(x)$ at $\omega_0, \omega_2, \dots, \omega_{2n-2}$,
- 2 evaluation of $A_{\text{odd}}(x)$ at $\omega_0, \omega_2, \dots, \omega_{2n-2}$,
- 3 n sums of $A_{\text{even}}(\omega_0) + \omega_0 A_{\text{odd}}(\omega_0), A_{\text{even}}(\omega_1) + \omega_1 A_{\text{odd}}(\omega_1), \dots, A_{\text{even}}(\omega_{2n-2}) + \omega_{n-1} A_{\text{odd}}(\omega_{2n-2})$
- 4 n sums of $A_{\text{even}}(\omega_0) + \omega_n A_{\text{odd}}(\omega_0), A_{\text{even}}(\omega_1) + \omega_{n-1} A_{\text{odd}}(\omega_1), \dots, A_{\text{even}}(\omega_{2n-2}) + \omega_{2n-1} A_{\text{odd}}(\omega_{2n-2})$

Denote by $T(2n)$ the time to evaluate $A(x)$ at the $2n$ roots of unity.

We have $T(2n) \leq 2T(n) + O(n)$ because

- 1 the size of $A_{\text{even}}(x)$ is n ,
- 2 the size of $A_{\text{odd}}(x)$ is n ,
- 3 taking n sums is $O(n)$,
- 4 taking n sums is $O(n)$.

$O(n \log(n))$ time

The recurrence $T(2n) \leq 2T(n) + O(n)$ is equivalent to $T(n) \leq 2T(n/2) + O(n)$.

So the following theorem applies:

Theorem (1)

If $T(n) \leq 2T(n/2) + c_1 n$ and $T(2) \leq c_2$, for positive constants c_1 and c_2 , then $T(n)$ is $O(n \log(n))$.

Fast Convolution; Strassen's Method

1 Fast Convolution

- reduction to subquadratic time
- polynomial evaluation at complex roots of unity
- interpolation via evaluation at complex roots of unity

2 The Master Method to Solve Recurrences

- a recursive matrix-matrix multiplication
- the master method

3 Strassen's method

- the steps in the method
- reduction to subcubic time

a property of complex roots of unity

Consider:

$$x^{2n} - 1 = (x - 1) \left(x^{2n-1} + \dots + x + 1 \right).$$

For any ω , a complex root of unity: $\omega^{2n} - 1 = 0$.

Consider $\omega \neq 1$: $x - 1 \neq 0$, as ω satisfies $x^{2n} - 1 = 0$, we must have that the other factor $x^{2n-1} + \dots + x + 1 = 0$.

Because $\omega^{2n} - 1 = 0$, we then have
for $\omega \neq 1$: $\omega^{2n-1} + \dots + \omega + 1 = 0$.

For $\omega = 1$, substitute 1 in $\omega^{2n-1} + \dots + \omega + 1 = \underbrace{1 + \dots + 1 + 1}_{2n} = 2n$.

To conclude:

$$\omega^{2n-1} + \dots + \omega + 1 = \begin{cases} 0 & \text{if } \omega \neq 1, \\ 2n & \text{if } \omega = 1. \end{cases}$$

interpolation by evaluation

Given $C(\omega_0), C(\omega_1), C(\omega_2), C(\omega_3), C(\omega_4), C(\omega_5), C(\omega_6), C(\omega_7)$, we want to compute coefficients $c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7$ of $C(x) = c_0 + c_1x + c_2x^2 + c_3x^3 + c_4x^4 + c_5x^5 + c_6x^6 + c_7x^7$.

$D(x) =$

$$\begin{aligned} & C(\omega_0) + C(\omega_1)x + C(\omega_2)x^2 + C(\omega_3)x^3 + C(\omega_4)x^4 + C(\omega_5)x^5 + C(\omega_6)x^6 + C(\omega_7)x^7 \\ &= (c_0 + c_1\omega_0 + c_2\omega_0^2 + c_3\omega_0^3 + c_4\omega_0^4 + c_5\omega_0^5 + c_6\omega_0^6 + c_7\omega_0^7)1 \\ &+ (c_0 + c_1\omega_1 + c_2\omega_1^2 + c_3\omega_1^3 + c_4\omega_1^4 + c_5\omega_1^5 + c_6\omega_1^6 + c_7\omega_1^7)x \\ &+ (c_0 + c_1\omega_2 + c_2\omega_2^2 + c_3\omega_2^3 + c_4\omega_2^4 + c_5\omega_2^5 + c_6\omega_2^6 + c_7\omega_2^7)x^2 \\ &+ (c_0 + c_1\omega_3 + c_2\omega_3^2 + c_3\omega_3^3 + c_4\omega_3^4 + c_5\omega_3^5 + c_6\omega_3^6 + c_7\omega_3^7)x^3 \\ &+ (c_0 + c_1\omega_4 + c_2\omega_4^2 + c_3\omega_4^3 + c_4\omega_4^4 + c_5\omega_4^5 + c_6\omega_4^6 + c_7\omega_4^7)x^4 \\ &+ (c_0 + c_1\omega_5 + c_2\omega_5^2 + c_3\omega_5^3 + c_4\omega_5^4 + c_5\omega_5^5 + c_6\omega_5^6 + c_7\omega_5^7)x^5 \\ &+ (c_0 + c_1\omega_6 + c_2\omega_6^2 + c_3\omega_6^3 + c_4\omega_6^4 + c_5\omega_6^5 + c_6\omega_6^6 + c_7\omega_6^7)x^6 \\ &+ (c_0 + c_1\omega_7 + c_2\omega_7^2 + c_3\omega_7^3 + c_4\omega_7^4 + c_5\omega_7^5 + c_6\omega_7^6 + c_7\omega_7^7)x^7 \end{aligned}$$

rearranging the coefficients

$$\begin{aligned} D(x) &= (c_0 + c_1\omega_0 + c_2\omega_0^2 + c_3\omega_0^3 + c_4\omega_0^4 + c_5\omega_0^5 + c_0\omega_0^6 + c_7\omega_0^7)1 \\ &+ (c_0 + c_1\omega_1 + c_2\omega_1^2 + c_3\omega_1^3 + c_4\omega_1^4 + c_5\omega_1^5 + c_1\omega_1^6 + c_7\omega_1^7)x \\ &+ (c_0 + c_1\omega_2 + c_2\omega_2^2 + c_3\omega_2^3 + c_4\omega_2^4 + c_5\omega_2^5 + c_2\omega_2^6 + c_7\omega_2^7)x^2 \\ &+ (c_0 + c_1\omega_3 + c_2\omega_3^2 + c_3\omega_3^3 + c_4\omega_3^4 + c_5\omega_3^5 + c_3\omega_3^6 + c_7\omega_3^7)x^3 \\ &+ (c_0 + c_1\omega_4 + c_2\omega_4^2 + c_3\omega_4^3 + c_4\omega_4^4 + c_5\omega_4^5 + c_4\omega_4^6 + c_7\omega_4^7)x^4 \\ &+ (c_0 + c_1\omega_5 + c_2\omega_5^2 + c_3\omega_5^3 + c_4\omega_5^4 + c_5\omega_5^5 + c_5\omega_5^6 + c_7\omega_5^7)x^5 \\ &+ (c_0 + c_1\omega_6 + c_2\omega_6^2 + c_3\omega_6^3 + c_4\omega_6^4 + c_5\omega_6^5 + c_6\omega_6^6 + c_7\omega_6^7)x^6 \\ &+ (c_0 + c_1\omega_7 + c_2\omega_7^2 + c_3\omega_7^3 + c_4\omega_7^4 + c_5\omega_7^5 + c_7\omega_7^6 + c_7\omega_7^7)x^7 \\ \\ &= (1 + x + x^2 + x^3 + x^4 + x^5 + x^6 + x^7)c_0 \\ &+ (\omega_0 + \omega_1x + \omega_2x^2 + \omega_3x^3 + \omega_4x^4 + \omega_5x^5 + \omega_6x^6 + \omega_7x^7)c_1 \\ &+ (\omega_0^2 + \omega_1^2x + \omega_2^2x^2 + \omega_3^2x^3 + \omega_4^2x^4 + \omega_5^2x^5 + \omega_6^2x^6 + \omega_7^2x^7)c_2 \\ &+ (\omega_0^3 + \omega_1^3x + \omega_2^3x^2 + \omega_3^3x^3 + \omega_4^3x^4 + \omega_5^3x^5 + \omega_6^3x^6 + \omega_7^3x^7)c_3 \\ &+ (\omega_0^4 + \omega_1^4x + \omega_2^4x^2 + \omega_3^4x^3 + \omega_4^4x^4 + \omega_5^4x^5 + \omega_6^4x^6 + \omega_7^4x^7)c_4 \\ &+ (\omega_0^5 + \omega_1^5x + \omega_2^5x^2 + \omega_3^5x^3 + \omega_4^5x^4 + \omega_5^5x^5 + \omega_6^5x^6 + \omega_7^5x^7)c_5 \\ &+ (\omega_0^6 + \omega_1^6x + \omega_2^6x^2 + \omega_3^6x^3 + \omega_4^6x^4 + \omega_5^6x^5 + \omega_6^6x^6 + \omega_7^6x^7)c_6 \\ &+ (\omega_0^7 + \omega_1^7x + \omega_2^7x^2 + \omega_3^7x^3 + \omega_4^7x^4 + \omega_5^7x^5 + \omega_6^7x^6 + \omega_7^7x^7)c_7 \end{aligned}$$

simplifying the powers of the roots of unity

$$\begin{aligned} D(x) = & (1 + x + x^2 + x^3 + x^4 + x^5 + x^6 + x^7)c_0 \\ & + (\omega_0 + \omega_1 x + \omega_2 x^2 + \omega_3 x^3 + \omega_4 x^4 + \omega_5 x^5 + \omega_6 x^6 + \omega_7 x^7)c_1 \\ & + (\omega_0^2 + \omega_1^2 x + \omega_2^2 x^2 + \omega_3^2 x^3 + \omega_4^2 x^4 + \omega_5^2 x^5 + \omega_6^2 x^6 + \omega_7^2 x^7)c_2 \\ & + (\omega_0^3 + \omega_1^3 x + \omega_2^3 x^2 + \omega_3^3 x^3 + \omega_4^3 x^4 + \omega_5^3 x^5 + \omega_6^3 x^6 + \omega_7^3 x^7)c_3 \\ & + (\omega_0^4 + \omega_1^4 x + \omega_2^4 x^2 + \omega_3^4 x^3 + \omega_4^4 x^4 + \omega_5^4 x^5 + \omega_6^4 x^6 + \omega_7^4 x^7)c_4 \\ & + (\omega_0^5 + \omega_1^5 x + \omega_2^5 x^2 + \omega_3^5 x^3 + \omega_4^5 x^4 + \omega_5^5 x^5 + \omega_6^5 x^6 + \omega_7^5 x^7)c_5 \\ & + (\omega_0^6 + \omega_1^6 x + \omega_2^6 x^2 + \omega_3^6 x^3 + \omega_4^6 x^4 + \omega_5^6 x^5 + \omega_6^6 x^6 + \omega_7^6 x^7)c_6 \\ & + (\omega_0^7 + \omega_1^7 x + \omega_2^7 x^2 + \omega_3^7 x^3 + \omega_4^7 x^4 + \omega_5^7 x^5 + \omega_6^7 x^6 + \omega_7^7 x^7)c_7 \end{aligned}$$

$$\begin{aligned} = & (1 + x + x^2 + x^3 + x^4 + x^5 + x^6 + x^7)c_0 \\ & + (1 + \omega_1 x + \omega_2 x^2 + \omega_3 x^3 + \omega_4 x^4 + \omega_5 x^5 + \omega_6 x^6 + \omega_7 x^7)c_1 \\ & + (1 + \omega_2 x + \omega_4 x^2 + \omega_6 x^3 + 1x^4 + \omega_2 x^5 + \omega_4 x^6 + \omega_6 x^7)c_2 \\ & + (1 + \omega_3 x + \omega_6 x^2 + \omega_1 x^3 + \omega_4 x^4 + \omega_7 x^5 + \omega_2 x^6 + \omega_5 x^7)c_3 \\ & + (1 + \omega_4 x + 1x^2 + \omega_4 x^3 + 1x^4 + \omega_4 x^5 + 1x^6 + \omega_4 x^7)c_4 \\ & + (1 + \omega_5 x + \omega_2 x^2 + \omega_7 x^3 + \omega_4 x^4 + \omega_1 x^5 + \omega_6 x^6 + \omega_3 x^7)c_5 \\ & + (1 + \omega_6 x + \omega_4 x^2 + \omega_2 x^3 + 1x^4 + \omega_6 x^5 + \omega_4 x^6 + \omega_2 x^7)c_6 \\ & + (1 + \omega_7 x + \omega_6 x^2 + \omega_5 x^3 + \omega_4 x^4 + \omega_3 x^5 + \omega_2 x^6 + \omega_1 x^7)c_7 \end{aligned}$$

simplifying more: $\omega_k = \omega_1^k, \omega_k^j = \omega_1^{k \cdot j} = \omega_1^{k \cdot j \bmod 2n}$

$$\begin{aligned}
 D(x) = & (1 + x + x^2 + x^3 + x^4 + x^5 + x^6 + x^7)c_0 \\
 & + (1 + \omega_1 x + \omega_2 x^2 + \omega_3 x^3 + \omega_4 x^4 + \omega_5 x^5 + \omega_6 x^6 + \omega_7 x^7)c_1 \\
 & + (1 + \omega_2 x + \omega_4 x^2 + \omega_6 x^3 + 1x^4 + \omega_2 x^5 + \omega_4 x^6 + \omega_6 x^7)c_2 \\
 & + (1 + \omega_3 x + \omega_6 x^2 + \omega_1 x^3 + \omega_4 x^4 + \omega_7 x^5 + \omega_2 x^6 + \omega_5 x^7)c_3 \\
 & + (1 + \omega_4 x + 1x^2 + \omega_4 x^3 + 1x^4 + \omega_4 x^5 + 1x^6 + \omega_4 x^7)c_4 \\
 & + (1 + \omega_5 x + \omega_2 x^2 + \omega_7 x^3 + \omega_4 x^4 + \omega_1 x^5 + \omega_6 x^6 + \omega_3 x^7)c_5 \\
 & + (1 + \omega_6 x + \omega_4 x^2 + \omega_2 x^3 + 1x^4 + \omega_6 x^5 + \omega_4 x^6 + \omega_2 x^7)c_6 \\
 & + (1 + \omega_7 x + \omega_6 x^2 + \omega_5 x^3 + \omega_4 x^4 + \omega_3 x^5 + \omega_2 x^6 + \omega_1 x^7)c_7
 \end{aligned}$$

$$\begin{aligned}
 = & (1 + x + x^2 + x^3 + x^4 + x^5 + x^6 + x^7)c_0 \\
 & + (1 + \omega_1 x + \omega_1^2 x^2 + \omega_1^3 x^3 + \omega_1^4 x^4 + \omega_1^5 x^5 + \omega_1^6 x^6 + \omega_1^7 x^7)c_1 \\
 & + (1 + \omega_2 x + \omega_2^2 x^2 + \omega_2^3 x^3 + \omega_2^4 x^4 + \omega_2^5 x^5 + \omega_2^6 x^6 + \omega_2^7 x^7)c_2 \\
 & + (1 + \omega_3 x + \omega_3^2 x^2 + \omega_3^3 x^3 + \omega_3^4 x^4 + \omega_3^5 x^5 + \omega_3^6 x^6 + \omega_3^7 x^7)c_3 \\
 & + (1 + \omega_4 x + \omega_4^2 x^2 + \omega_4^3 x^3 + \omega_4^4 x^4 + \omega_4^5 x^5 + \omega_4^6 x^6 + \omega_4^7 x^7)c_4 \\
 & + (1 + \omega_5 x + \omega_5^2 x^2 + \omega_5^3 x^3 + \omega_5^4 x^4 + \omega_5^5 x^5 + \omega_5^6 x^6 + \omega_5^7 x^7)c_5 \\
 & + (1 + \omega_6 x + \omega_6^2 x^2 + \omega_6^3 x^3 + \omega_6^4 x^4 + \omega_6^5 x^5 + \omega_6^6 x^6 + \omega_6^7 x^7)c_6 \\
 & + (1 + \omega_7 x + \omega_7^2 x^2 + \omega_7^3 x^3 + \omega_7^4 x^4 + \omega_7^5 x^5 + \omega_7^6 x^6 + \omega_7^7 x^7)c_7
 \end{aligned}$$

evaluation at the roots of unity

$$\begin{aligned}
 D(x) = & (1 + x + x^2 + x^3 + x^4 + x^5 + x^6 + x^7)c_0 \\
 & + (1 + \omega_1 x + \omega_1^2 x^2 + \omega_1^3 x^3 + \omega_1^4 x^4 + \omega_1^5 x^5 + \omega_1^6 x^6 + \omega_1^7 x^7)c_1 \\
 & + (1 + \omega_2 x + \omega_2^2 x^2 + \omega_2^3 x^3 + \omega_2^4 x^4 + \omega_2^5 x^5 + \omega_2^6 x^6 + \omega_2^7 x^7)c_2 \\
 & + (1 + \omega_3 x + \omega_3^2 x^2 + \omega_3^3 x^3 + \omega_3^4 x^4 + \omega_3^5 x^5 + \omega_3^6 x^6 + \omega_3^7 x^7)c_3 \\
 & + (1 + \omega_4 x + \omega_4^2 x^2 + \omega_4^3 x^3 + \omega_4^4 x^4 + \omega_4^5 x^5 + \omega_4^6 x^6 + \omega_4^7 x^7)c_4 \\
 & + (1 + \omega_5 x + \omega_5^2 x^2 + \omega_5^3 x^3 + \omega_5^4 x^4 + \omega_5^5 x^5 + \omega_5^6 x^6 + \omega_5^7 x^7)c_5 \\
 & + (1 + \omega_6 x + \omega_6^2 x^2 + \omega_6^3 x^3 + \omega_6^4 x^4 + \omega_6^5 x^5 + \omega_6^6 x^6 + \omega_6^7 x^7)c_6 \\
 & + (1 + \omega_7 x + \omega_7^2 x^2 + \omega_7^3 x^3 + \omega_7^4 x^4 + \omega_7^5 x^5 + \omega_7^6 x^6 + \omega_7^7 x^7)c_7
 \end{aligned}$$

$$\omega^{2n-1} + \dots + \omega + 1 = \begin{cases} 0 & \text{if } \omega \neq 1, \\ 2n & \text{if } \omega = 1, \end{cases} \text{ implies } \begin{cases} D(\omega_0) = 8c_0 \\ D(\omega_1) = 8c_7 \\ D(\omega_2) = 8c_6 \\ D(\omega_3) = 8c_5 \\ D(\omega_4) = 8c_4 \\ D(\omega_5) = 8c_3 \\ D(\omega_6) = 8c_2 \\ D(\omega_7) = 8c_1 \end{cases}$$

the general form of interpolation by evaluation

$$\text{Let } D(x) = \sum_{k=0}^{2n-1} C(\omega_k) x^k = \sum_{k=0}^{2n-1} \left(\sum_{\ell=0}^{2n-1} c_{\ell} \omega_k^{\ell} \right) x^k.$$

$$\text{Then } D(\omega_j) = \sum_{k=0}^{2n-1} \left(\sum_{\ell=0}^{2n-1} c_{\ell} \omega_k^{\ell} \right) \omega_j^k = \sum_{\ell=0}^{2n-1} c_{\ell} \left(\sum_{k=0}^{2n-1} \omega_k^{\ell} \omega_j^k \right),$$

$$\text{and } D(\omega_j) = \sum_{\ell=0}^{2n-1} c_{\ell} \left(\sum_{k=0}^{2n-1} e^{(2\pi i/2n)(k \cdot \ell + j \cdot k)} \right) = \sum_{\ell=0}^{2n-1} c_{\ell} \left(\sum_{k=0}^{2n-1} \omega_k^{\ell+j} \right).$$

$$\omega^{2n-1} + \dots + \omega + 1 = \begin{cases} 0 & \text{if } \omega \neq 1, \\ 2n & \text{if } \omega = 1, \end{cases} \text{ implies } D(\omega_k) = 2n c_{2n-k},$$

$$\text{for } k = 0, 1, \dots, 2n-1. \text{ Or equivalently, } c_k = \frac{D(\omega_{2n-k})}{2n}.$$

a recursive step 3

For $D(x) = \sum_{k=0}^{2n-1} C(\omega_k)x^k$, we have

$$c_k = \frac{D(\omega_{2n-k})}{2n}, \quad k = 0, 1, \dots, 2n-1.$$

We evaluate a polynomial at complex roots of unity, so the same recurrence $T(2n) \leq T(n) + O(n)$ applies and we obtain a subquadratic, $O(n \log(n))$ algorithm to compute the coefficients of the convolution.

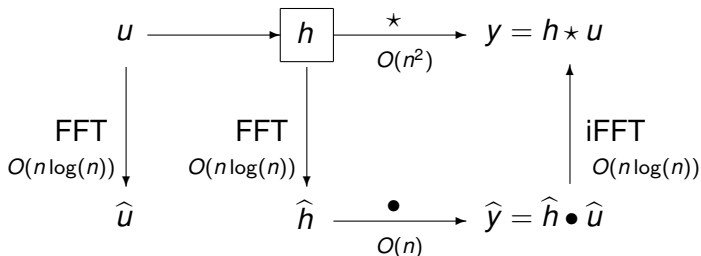
Theorem (2)

For $a = (a_0, a_1, \dots, a_{n-1})$ and $b = (b_0, b_1, \dots, b_{n-1})$, the convolution $c = a \star b$ can be computed in $O(n \log(n))$ time.

The algorithm using the complex roots of unity is known as *the Fast Fourier Transform*, abbreviated as FFT.

the convolution theorem applied to a filter with the FFT

The FFT turns the convolution $\star$ into the componentwise product $\bullet$ transforming signals from the time and the frequency domain. The inverse FFT is denoted by the iFFT.



With the FFT, the convolution of two n -vectors is $O(n \log(n))$.

permute and partition the Fourier matrix $F_n = \frac{1}{\sqrt{n}}M_n$

$$M_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega & \omega^2 & \omega^3 \\ 1 & \omega^2 & 1 & \omega^2 \\ 1 & \omega^3 & \omega^2 & \omega \end{bmatrix} \quad P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Multiplying M_4 with P swaps columns 2 and 3.

$$M_4P = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega^2 & \omega & \omega^3 \\ 1 & 1 & \omega^2 & \omega^2 \\ 1 & \omega^2 & \omega^3 & \omega \end{bmatrix} = \left[\begin{array}{cc|cc} 1 & 1 & 1 & 1 \\ 1 & \omega^2 & \omega & \omega^3 \\ 1 & 1 & \omega^2 & \omega^2 \\ 1 & \omega^2 & \omega^3 & \omega \end{array} \right]$$

Observe the repeated occurrence of M_2 :

$$M_2 = \begin{bmatrix} 1 & 1 \\ 1 & \omega^2 \end{bmatrix}, \quad F_2 = \frac{1}{\sqrt{2}}M_2.$$

the butterfly matrix

With

$$M_2 = \begin{bmatrix} 1 & 1 \\ 1 & \omega^2 \end{bmatrix} \quad \text{and} \quad D_2 = \begin{bmatrix} 1 & 0 \\ 0 & \omega \end{bmatrix}$$

we rewrite $M_4 P$ as

$$\left[\begin{array}{cc|cc} 1 & 1 & 1 & 1 \\ 1 & \omega^2 & \omega & \omega^3 \\ \hline 1 & 1 & \omega^2 & \omega^2 \\ 1 & \omega^2 & \omega^3 & \omega \end{array} \right] = \begin{bmatrix} M_2 & D_2 M_2 \\ M_2 & -D_2 M_2 \end{bmatrix} = \begin{bmatrix} I_2 & D_2 \\ I_2 & -D_2 \end{bmatrix} \begin{bmatrix} M_2 & 0 \\ 0 & M_2 \end{bmatrix}.$$

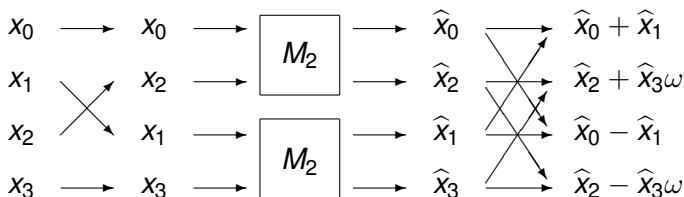
The matrix

$$B_4 = \begin{bmatrix} I_2 & D_2 \\ I_2 & -D_2 \end{bmatrix}, \quad I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

is called *the butterfly matrix*.

the butterfly picture

$$B_4 \begin{bmatrix} \hat{x}_0 \\ \hat{x}_2 \\ \hat{x}_1 \\ \hat{x}_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & \omega \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -\omega \end{bmatrix} \begin{bmatrix} \hat{x}_0 \\ \hat{x}_2 \\ \hat{x}_1 \\ \hat{x}_3 \end{bmatrix} = \begin{bmatrix} \hat{x}_0 + \hat{x}_1 \\ \hat{x}_2 + \hat{x}_3\omega \\ \hat{x}_0 - \hat{x}_1 \\ \hat{x}_2 - \hat{x}_3\omega \end{bmatrix}$$



reformulate step 3 with matrices

Exercise 3:

The computation of the coefficients of C on slide #17 from the values $C(x_0), C(x_1), \dots, C(x_{2n-1})$ can be expressed as a matrix-vector product:

- 1 The input of this product are the coefficients of C , and
- 2 the output are the values $C(x_0), C(x_1), \dots, C(x_{2n-1})$.

Write the matrix and its inverse for $n = 4$ and $n = 8$.

Does the structure of the matrix show the reduction to subquadratic time?

Fast Convolution; Strassen's Method

1 Fast Convolution

- reduction to subquadratic time
- polynomial evaluation at complex roots of unity
- interpolation via evaluation at complex roots of unity

2 The Master Method to Solve Recurrences

- a recursive matrix-matrix multiplication
- the master method

3 Strassen's method

- the steps in the method
- reduction to subcubic time

CS 401/MCS 401 Lecture 9
Computer Algorithms I
Jan Verschelde, 8 July 2026

Fast Convolution; Strassen's Method

1 Fast Convolution

- reduction to subquadratic time
- polynomial evaluation at complex roots of unity
- interpolation via evaluation at complex roots of unity

2 The Master Method to Solve Recurrences

- a recursive matrix-matrix multiplication
- the master method

3 Strassen's method

- the steps in the method
- reduction to subcubic time

a recursive matrix-matrix multiplication

Given are two n -by- n matrices A and B ,
we want to compute their product $C = A \star B$.

Partition each matrix into four $n/2$ -by- $n/2$ -matrices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}.$$

Then the product of A and B is

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} \star B_{11} + A_{12} \star B_{21} & A_{11} \star B_{12} + A_{12} \star B_{22} \\ A_{21} \star B_{11} + A_{22} \star B_{21} & A_{21} \star B_{12} + A_{22} \star B_{22} \end{bmatrix}.$$

One matrix multiplication $A \star B$ on an n -by- n matrix
is replaced by 8 matrix multiplications on $n/2$ -by- $n/2$ matrices,
and 4 additions of $n/2$ -by- $n/2$ matrices.

setting up the recurrence

Let $T(n)$ be the time to multiply two n -by- n matrices.

The base case for the recursion is $n = 1$: $c_{11} = a_{11} \cdot b_{11}$, which runs in $\Theta(1)$, constant time.

For $n > 1$:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} \star B_{11} + A_{12} \star B_{21} & A_{11} \star B_{12} + A_{12} \star B_{22} \\ A_{21} \star B_{11} + A_{22} \star B_{21} & A_{21} \star B_{12} + A_{22} \star B_{22} \end{bmatrix}.$$

The addition of two matrices of dimension n costs $\Theta(n^2)$.

Thus, we have the recurrence:

$$T(n) = 8T(n/2) + \Theta(n^2), \quad \text{for } n > 1.$$

Fast Convolution; Strassen's Method

1 Fast Convolution

- reduction to subquadratic time
- polynomial evaluation at complex roots of unity
- interpolation via evaluation at complex roots of unity

2 The Master Method to Solve Recurrences

- a recursive matrix-matrix multiplication
- the master method

3 Strassen's method

- the steps in the method
- reduction to subcubic time

Theorem (3, the master theorem)

The recurrence relation

$$T(n) = aT(n/b) + f(n), \quad a \geq 1, b > 1, \text{ some function } f$$

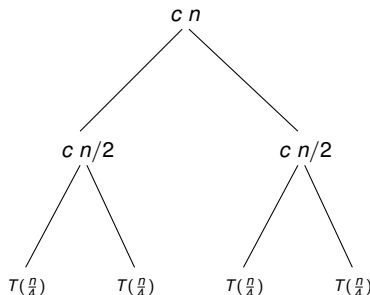
has the following bounds:

- 1 If $f(n)$ is $O(n^{\log_b(a)-\epsilon})$ for some constant $\epsilon > 0$, then $T(n)$ is $\Theta(n^{\log_b(a)})$.
- 2 If $f(n)$ is $\Theta(n^{\log_b(a)})$, then $T(n)$ is $\Theta(n^{\log_b(a)} \log_2(n))$.
- 3 If $f(n)$ is $\Omega(n^{\log_b(a)+\epsilon})$, for some constant $\epsilon > 0$, and

$$a f(n/b) \leq c f(n), \text{ for some constant } c < 1 \text{ and } n \geq N,$$

then $T(n)$ is $\Theta(f(n))$.

unrolling the recurrence $T(n) = 2T(n/2) + cn$



In this specific case (the middle case of the theorem), the total cost is evenly distributed among the levels in the tree.

In the two other cases, the total cost is

- ❶ either dominated by the cost at the leaves,
- ❷ or dominated by the cost at the root.

interpretation of the theorem

Recall: the recurrence $T(n) = aT(n/b) + f(n)$ defines a tree of depth $\log_b(n)$ with $L = n^{\log_b(a)}$ leaves.

Three cases:

- 1 f grows no faster than L : f is $O(n^{\log_b(a)})$
- 2 f grows at the same rate as L : f is $\Theta(n^{\log_b(a)})$
- 3 f grows at least as fast as L : f is $\Omega(n^{\log_b(a)})$

Relating $f(n)$ to L , we find that $T(n)$ depends on L .

Second case: f is $\Theta(L) \Rightarrow T(n)$ is $\Theta(L \log_2(n))$.

Note: depth $\log_b(n) = \log_2(n) / \log_2(b)$ is $\Theta(\log_2(n))$.

For a proof of the theorem, read section 4.7 of the book by Thomas H. Cormen, Charles E. Leieron, Ronald L. Rivest, Clifford Stein: *Introduction to Algorithms*, Third edition, the MIT Press, 2009.

an illustration: merge sort

The merge sort algorithm has a recurrence

$$T(n) = 2T(n/2) + \Theta(n),$$

where $a = 2$, $b = 2$, and $f(n) = \Theta(n)$.

The number of leaves: $L = n^{\log_b(a)} = n^{\log_2(2)} = n$,
so case 2 of the theorem applies:

2 If $f(n)$ is $\Theta(n^{\log_b(a)})$, then $T(n)$ is $\Theta(n^{\log_b(a)} \log_2(n))$.

Therefore: $T(n)$ is $\Theta(n \log_2(n))$.

applied to recursive matrix-matrix multiplication

The recursive matrix-matrix multiplication has the recurrence

$$T(n) = 8T(n/2) + \Theta(n^2),$$

where $a = 8$, $b = 2$, and $f(n) = \Theta(n^2)$.

Compute $n^{\log_b(a)} = n^{\log_2(8)} = n^3$,
so case 1 of the theorem applies:

- 1 If $f(n)$ is $O(n^{\log_b(a)-\epsilon})$ for some constant $\epsilon > 0$,
then $T(n)$ is $\Theta(n^{\log_b(a)})$.

Take $\epsilon = 1$. Therefore: $T(n)$ is $\Theta(n^3)$.

applying the master method

Exercise 4:

Use the master method to give a tight asymptotic bound on the recurrence

$$T(n) = 2T(n/4) + \sqrt{n}.$$

Exercise 5:

Solve the recurrence

$$T(n) = 4T(n/2) + 1.$$

Show how you arrived at your solution.

Fast Convolution; Strassen's Method

1 Fast Convolution

- reduction to subquadratic time
- polynomial evaluation at complex roots of unity
- interpolation via evaluation at complex roots of unity

2 The Master Method to Solve Recurrences

- a recursive matrix-matrix multiplication
- the master method

3 Strassen's method

- the steps in the method
- reduction to subcubic time

steps 1 and 2 of Strassen's method

Given are two n -by- n matrices A and B ,
we want to compute their product $C = A \star B$.

- 1 Partition each matrix into four $n/2$ -by- $n/2$ -matrices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}.$$

- 2 Make the following 10 matrices

$$\begin{array}{ll} S_1 &= B_{12} - B_{22}, & S_2 &= A_{11} + A_{12}, \\ S_3 &= A_{21} + A_{22}, & S_4 &= B_{21} - B_{11}, \\ S_5 &= A_{11} + A_{22}, & S_6 &= B_{11} + B_{22}, \\ S_7 &= A_{12} - A_{22}, & S_8 &= B_{21} + B_{22}, \\ S_9 &= A_{11} - A_{21}, & S_{10} &= B_{11} + B_{12}. \end{array}$$

This step takes $\Theta(n^2)$ time.

steps 3 and 4 of Strassen's method

- 3 Multiply seven times $n/2$ -by- $n/2$ matrices:

$$P_1 = A_{11} \star S_1 = A_{11} \star B_{12} - A_{11} \star B_{22},$$

$$P_2 = S_2 \star B_{22} = A_{11} \star B_{22} + A_{12} \star B_{22},$$

$$P_3 = S_3 \star B_{11} = A_{21} \star B_{11} + A_{22} \star B_{11},$$

$$P_4 = A_{22} \star S_4 = A_{22} \star B_{21} - A_{22} \star B_{11},$$

$$P_5 = S_5 \star S_6 = A_{11} \star B_{11} + A_{11} \star B_{22} + A_{22} \star B_{11} + A_{22} \star B_{22},$$

$$P_6 = S_7 \star S_8 = A_{12} \star B_{21} + A_{12} \star B_{22} - A_{22} \star B_{21} - A_{22} \star B_{22},$$

$$P_7 = S_9 \star S_{10} = A_{11} \star B_{11} + A_{11} \star B_{12} - A_{12} \star B_{11} - A_{21} \star B_{12}.$$

- 4 Construct the matrices in the product C :

$$C_{11} = P_5 + P_4 - P_2 + P_6,$$

$$C_{12} = P_1 + P_2,$$

$$C_{21} = P_3 + P_4,$$

$$C_{22} = P_5 + P_1 - P_3 - P_7.$$

verifying correctness for C_{12}

Strassen's method defines $C_{12} = P_1 + P_2$, with

$$\begin{aligned}P_1 &= A_{11} \star S_1 = A_{11} \star B_{12} - A_{11} \star B_{22}, \\P_2 &= S_2 \star B_{22} = A_{11} \star B_{22} + A_{12} \star B_{22},\end{aligned}$$

With direct verification:

$$\begin{array}{rcccl} & A_{11} \star B_{12} & - & A_{11} \star B_{22} & \\ + & & & A_{11} \star B_{22} & + A_{12} \star B_{22} \\ \hline & A_{11} \star B_{12} & & & + A_{12} \star B_{22} \end{array}$$

Indeed, we computed earlier: $C_{12} = A_{11} \star B_{12} + A_{12} \star B_{22}$.

verifying correctness for C_{21}

Strassen's method defines $C_{21} = P_3 + P_4$, with

$$\begin{aligned}P_3 &= S_3 \star B_{11} = A_{21} \star B_{11} + A_{22} \star B_{11}, \\P_4 &= A_{22} \star S_4 = A_{22} \star B_{21} - A_{22} \star B_{11},\end{aligned}$$

With direct verification:

$$\begin{array}{rcccl} & A_{21} \star B_{11} & + & A_{22} \star B_{11} & \\ + & & & - A_{22} \star B_{11} & + A_{22} \star B_{21} \\ \hline & A_{21} \star B_{11} & & & + A_{22} \star B_{21} \end{array}$$

Indeed, we computed earlier: $C_{21} = A_{21} \star B_{11} + A_{22} \star B_{21}$.

verifying correctness for C_{11}

Strassen's method defines $C_{11} = P_5 + P_4 - P_2 + P_6$, with

$$P_5 = A_{11} \star B_{11} + A_{11} \star B_{22} + A_{22} \star B_{11} + A_{22} \star B_{22},$$

$$P_4 = A_{22} \star B_{21} - A_{22} \star B_{11},$$

$$P_2 = A_{11} \star B_{22} + A_{12} \star B_{22},$$

$$P_6 = A_{12} \star B_{21} + A_{12} \star B_{22} - A_{22} \star B_{21} - A_{22} \star B_{22}.$$

With direct verification:

$$\begin{array}{r} A_{11} \star B_{11} + A_{11} \star B_{22} + A_{22} \star B_{11} + A_{22} \star B_{22} \\ \quad - A_{22} \star B_{11} \qquad \qquad \qquad + A_{22} \star B_{21} \\ \quad - A_{11} \star B_{22} \qquad \qquad \qquad + A_{12} \star B_{22} \\ + \qquad \qquad \qquad - A_{22} \star B_{22} - A_{22} \star B_{21} + A_{12} \star B_{22} + A_{12} \star B_{21} \\ \hline A_{11} \star B_{11} \qquad \qquad \qquad + A_{12} \star B_{21} \end{array}$$

Indeed, we computed earlier: $C_{11} = A_{11} \star B_{11} + A_{12} \star B_{21}$.

verifying correctness for C_{22}

Strassen's method defines $C_{22} = P_5 + P_1 - P_3 - P_7$, with

$$P_5 = A_{11} \star B_{11} + A_{11} \star B_{22} + A_{22} \star B_{11} + A_{22} \star B_{22},$$

$$P_1 = A_{11} \star B_{12} - A_{11} \star B_{22},$$

$$P_3 = A_{21} \star B_{11} + A_{22} \star B_{11},$$

$$P_7 = A_{11} \star B_{11} + A_{11} \star B_{12} - A_{12} \star B_{11} - A_{21} \star B_{12}.$$

With direct verification:

$$\begin{array}{r}
 A_{11} \star B_{11} + A_{11} \star B_{22} + A_{22} \star B_{11} + A_{22} \star B_{22} \\
 \quad - A_{11} \star B_{22} \qquad \qquad \qquad + A_{11} \star B_{12} \\
 \qquad \qquad \qquad - A_{22} \star B_{11} \qquad \qquad \qquad - A_{21} \star B_{11} \\
 + \quad - A_{11} \star B_{11} \qquad \qquad \qquad - A_{11} \star B_{12} + A_{21} \star B_{11} + A_{21} \star B_{12} \\
 \hline
 \qquad \qquad \qquad A_{22} \star B_{22} \qquad \qquad \qquad + A_{21} \star B_{12}
 \end{array}$$

Indeed, we computed earlier: $C_{22} = A_{21} \star B_{12} + A_{22} \star B_{22}$.

Fast Convolution; Strassen's Method

1 Fast Convolution

- reduction to subquadratic time
- polynomial evaluation at complex roots of unity
- interpolation via evaluation at complex roots of unity

2 The Master Method to Solve Recurrences

- a recursive matrix-matrix multiplication
- the master method

3 Strassen's method

- the steps in the method
- reduction to subcubic time

a recursive matrix multiplication (Strassen)

SMM(A, B, n)

Input: $A, B \in \mathbb{R}^{n \times n}$, n is power of 2.

Output: C , $C = A \star B$.

if $n = 1$ then

 return $A \cdot B$

else

$$\text{let } A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$S_1 := B_{12} - B_{22}; \quad S_2 := A_{11} + A_{12}$$

$$S_3 := A_{21} + A_{22}; \quad S_4 := B_{21} - B_{11}$$

$$S_5 := A_{11} + A_{22}; \quad S_6 := B_{11} + B_{22}$$

$$S_7 := A_{12} - A_{22}; \quad S_8 := B_{21} + B_{22}$$

$$S_9 := A_{11} - A_{21}; \quad S_{10} := B_{11} + B_{12}$$

the algorithm SMM continued

$$P_1 := \text{SMM}(A_{11}, S_1)$$

$$P_2 := \text{SMM}(S_2, B_{22})$$

$$P_3 := \text{SMM}(S_3, B_{11})$$

$$P_4 := \text{SMM}(A_{22}, S_4)$$

$$P_5 := \text{SMM}(S_5, S_6)$$

$$P_6 := \text{SMM}(S_7, S_8)$$

$$P_7 := \text{SMM}(S_9, S_{10})$$

$$C_{11} := P_5 + P_4 - P_2 + P_6$$

$$C_{12} := P_1 + P_2$$

$$C_{21} := P_3 + P_4$$

$$C_{22} := P_5 + P_1 - P_3 - P_7$$

$$\text{return } \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}.$$

reduction to subcubic time

Let $T(n)$ be the time of Strassen's method. We have the recurrence

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 7T(n/2) + \Theta(n^2) & \text{if } n > 1. \end{cases}$$

where $a = 7$, $b = 2$, and $f(n) = \Theta(n^2)$.

Compute $n^{\log_b(a)} = n^{\log_2(7)}$,

so case 1 of the theorem applies:

- 1 If $f(n)$ is $\Theta(n^{\log_b(a)-\epsilon})$ for some constant $\epsilon > 0$, then $T(n)$ is $\Theta(n^{\log_b(a)})$.

Take $\epsilon = 0.80$, as $\log_2(7) \approx 2.8074$

Therefore: $T(n)$ is $\Theta(n^{\log_2(7)}) = \Theta(n^{2.8074})$.

The exposition on the slides follows section 4.2 of the book by Thomas H. Cormen, Charles E. Leieron, Ronald L. Rivest, Clifford Stein: *Introduction to Algorithms*, Third edition, the MIT Press, 2009.

back to complex arithmetic

Exercise 6:

The multiplication of two complex numbers, $I = \sqrt{-1}$:

$$(a + bI) \times (c + dI) = (a \cdot c - b \cdot d) + (a \cdot d + b \cdot c)I$$

requires four multiplications of real numbers.

- 1 Show that the complex multiplication can be done with three real multiplications, at the expense of extra additions and subtractions.
- 2 Explain the generalization to multiplying complex matrices.

overview of divide and conquer algorithms

We close chapter 5 on Divide and Conquer.

The algorithms we covered are listed below.

- 1 counting inversions in a sequence
- 2 finding the closest pair of points
- 3 Karatsuba's integer multiplication
- 4 the fast Fourier transform
- 5 Strassen's method to multiply matrices

The divide and conquer algorithms improved the asymptotic costs of the straightforward algorithms from quadratic and cubic to subquadratic and subcubic times.